

Shield&Shot

두 손으로 쓰고 막는 하이퍼 캐주얼 물리 액션 게임

팀 프로젝트 결과보고서

훈련기관명 넥스트러너스

팀명 7팀 - Conqueror

팀원 오시진(팀장), 엄기태, 김현준, 최현호, 이찬수, 임기현

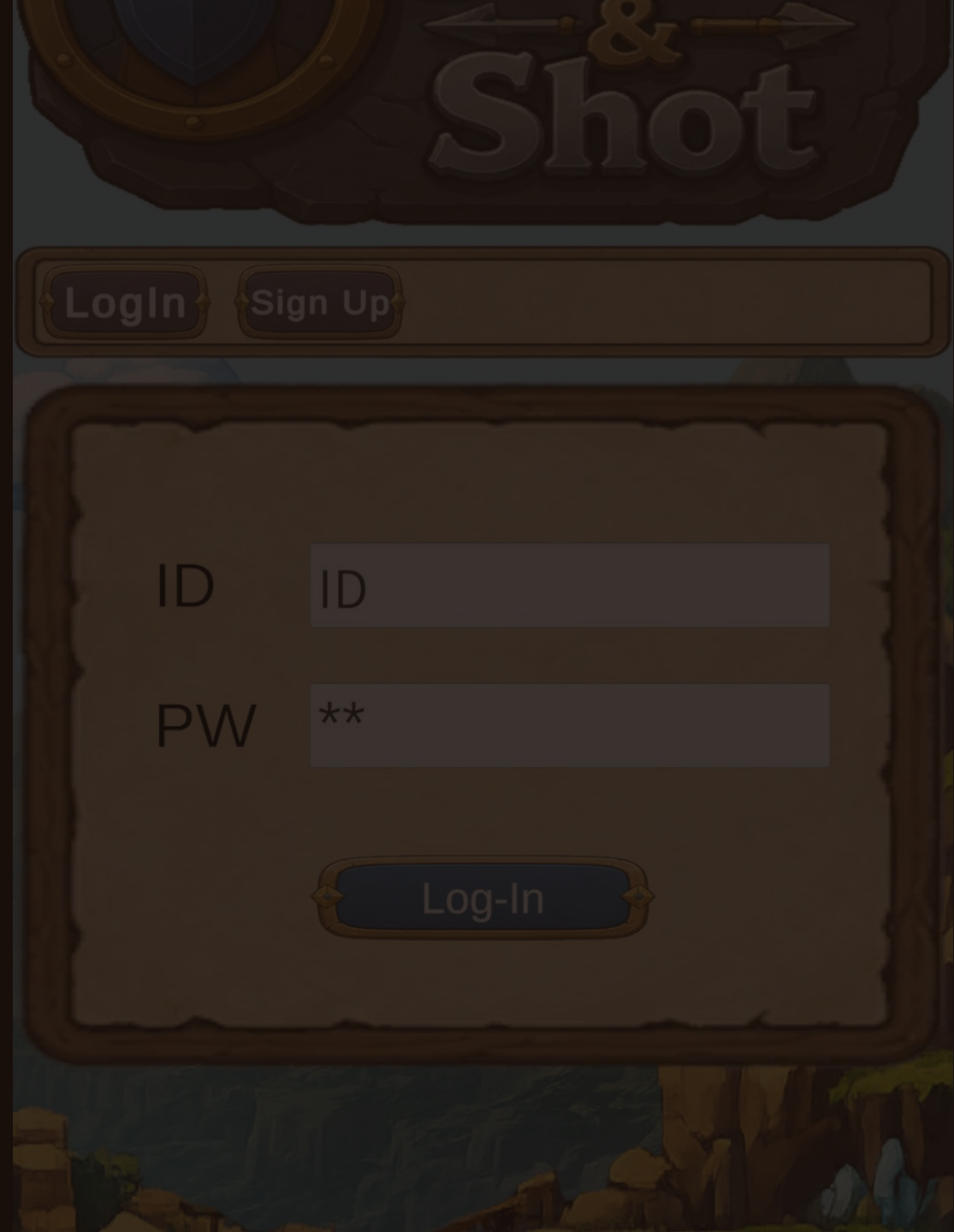
담당 멘토 이순현

Unity 6.3 LTS

Photon

뒤끝 Backend

UVCS / Notion



발표 흐름

01

프로젝트 개요

기획 의도, 게임 콘셉트, 기술 스택

02

팀 구성 및 역할

파트별 담당 업무와 협업 구조

03

수행 절차 및 일정

WBS 기반 개발 흐름

04

수행 결과

양손 조작, 증강, 필드 반응, 몬스터, PvP, Backend

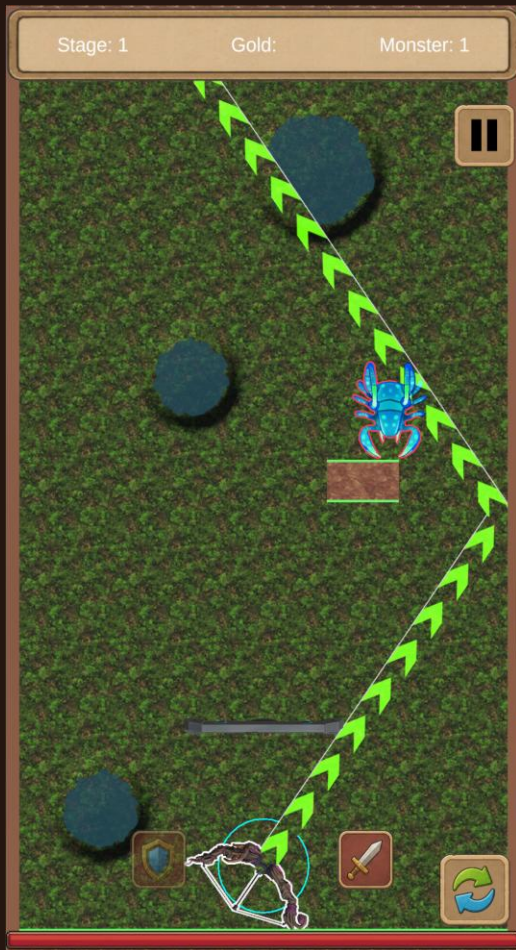
05

자체 평가 의견

완성도 평가와 개선 방향



두 손으로 쏘고 막는 물리 액션 디펜스



게임 소개

Shield&Shot은 모바일 세로 화면에서 왼손은 방패, 오른손은 무기를 조작하며 적의 공격을 막고 반격하는 하이퍼 캐주얼 물리 액션 게임입니다.

물리 액션

장르

Portrait

화면

Win / Android

플랫폼

핵심 차별점

- 공격/방어 영역을 분리한 양손 조작
- 투사체 반사, 조준선, 차징 기반의 손맛
- 셀 기반 맵과 속성 조합에 따른 환경 반응
- Photon 기반 PvP 전투 환경

기술 스택은 게임 루프와 온라인 기능을 중심으로 구성했다



Engine

Unity 6.3 LTS
6000.3.6f1



Language

C#



Network

Photon / Fusion
PvP 동기화



Backend

뒤끝 서버
로그인·뽑기·데이터



Collaboration

UVCS / Notion
WBS 일정관리

미구현 및 향후 확장 범위

랭킹 시스템, OAuth/간편로그인, MMR 레이팅/경쟁전, 실제 인앱결제, 출시용 과금/광고 완성 플로우는 기획 범위로 남기고 이번 결과물에서는 제외했습니다.

파트별 책임을 나누고 시스템 간 연결 지점을 맞췄다

오시진

팀장 / PM

전체 기획·시스템 설계, 양손 조작, 네트워크, 무기·증강·필드 구조

엄기태

전투 구현

WeaponCore, 무기/방패/증강 구현, VFX·SFX 연동

김현준

UI / Scene

UI 전체 구조, 뒤끝-UI 연동, 로딩씬과 씬 전환 처리

최현호

Monster

동적 난이도 스폰, 몬스터 특성·행동 주입, 보스/일반 몬스터

이찬수

Backend

뒤끝 로그인, 뽑기, 인벤토리, 장착 데이터 동기화

임기현

QA

테스트 케이스 작성, 기능 검증, 이슈 확인

06/04~07/07 동안 기획, 구현, 통합, QA를 병렬로 진행했다



06/04~06/09

사전 기획

아이디어 선정, 게임 콘셉트, 역할 분담

06/10~06/14

구조 설계

전투 루프, 데이터, 씬 플로우, Backend 구조

06/15~06/24

핵심 구현

증강, PvP, 필드, 몬스터, UI 기반 구축

06/25~07/03

통합

무기/방패/몬스터 /Backend/UI 연동

07/04~07/07

안정화

QA, 시연자료, 발표자료 정리

Scene Flow 시스템	SceneFlowManager	공통 전 전환 구조	BaseSceneController 기반 전 행동추가 설계	BaseSceneController.cs	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
			payload 기반 전 데이터 전달 구조 설계	SceneTransitionData.cs	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
			전 차단 및 전환 사유 정의	SceneType.cs	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
			전 진입이탈 초기화 규칙 표준화	SceneTransitionReason.cs	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
			전 컨트롤러 상속 구조 설계서		모시진	완료	100%	2026. 6. 17	2026. 6. 19	
			전 로드 공통 API 구현	SceneFlowManager.cs	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
			다음 전 payload 지점시켜 처리	SceneFlowManager.cs, SceneTransitionData.cs	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
			승격 후입 방지 및 로직 상태 관리	SceneFlowManager.cs	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
		씬별 Controller	Title 전 컨트롤러 제작	TitleSceneController.cs	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
			Lobby 전 컨트롤러 제작	LobbySceneController.cs	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
			Intro 전 컨트롤러 제작	IntroSceneController.cs	모시진	완료	100%	2026. 6. 18	2026. 6. 19	
			Indame 전 컨트롤러 제작	IndameSceneController.cs	모시진	완료	100%	2026. 6. 18	2026. 6. 19	
	Photon/Fusion 연결		PVP 전 컨트롤러 제작	PvpSceneController.cs	모시진	완료	100%	2026. 6. 18	2026. 6. 19	
			각 전별 payload 수신 및 초기화 처리	각 전 Controller 초기화 로직	모시진	완료	100%	2026. 6. 18	2026. 6. 19	
			NetworkRunner 프리팹 구성	NetworkRunner_QuickMatch.prefab	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
			Fusion callback 등록 및 Runner 행동추가 관리	NetworkMatchManager.cs	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
		Quick Match	서버 연결/종료/실제 처리	NetworkMatchManager.cs	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
			매치메이킹 버튼 UI 연동	MatchmakingPanelUI.cs, 매칭 버튼 오브젝트	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
			빠른 매칭 시작/최소 로직 구현	NetworkMatchManager.cs	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
			2명 입장 금지 처리	NetworkMatchManager_OnPlayerJoined()	모시진	완료	100%	2026. 6. 17	2026. 6. 19	
			매칭 성공 이벤트 발행	QuickMatchSucceeded 이벤트	모시진	완료	100%	2026. 6. 17	2026. 6. 19	

게임 루프는 조작, 전투, 성장, 데이터 저장으로 이어진다

로비

로그인 / 재화 / 메뉴

전투 진입

씬 전환 / 장착 데이터
로드

양손 조작

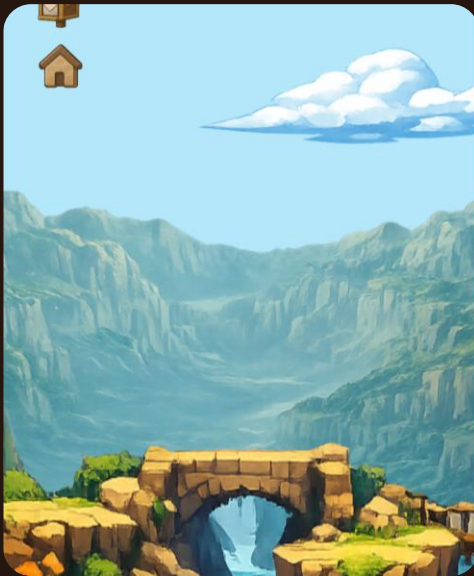
방패 방어 + 무기 조준

전투 결과

몬스터 처치 / 보스 /
PvP

성장

증강 / 뽑기 / 인벤토리



공격과 방어를 분리한 양손 조작으로 전투의 손맛을 만들었다



구현 포인트

- 왼쪽 방패 영역과 오른쪽 무기 영역 분리
- 드래그 기반 조준선과 발사 세기 조절
- 방패 회전/방어 판정과 무기 입력 동시 처리
- 모바일 환경의 직관적 조작 흐름 검증

투사체와 증강은 전투 결과를 매 웨이브 다르게 만든다



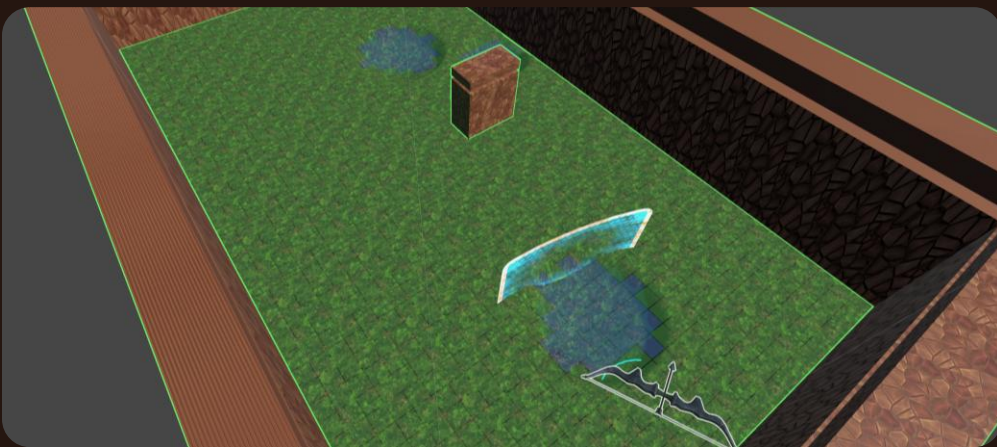
투사체 시스템

조준선, 반사 경로, 차징, 투사체 풀링을 연결해 발사부터 충돌까지의 생명 주기를 구성했습니다.

증강 시스템

선택한 증강을 플레이어 상태와 투사체 Behavior에 주입해 공격 방식과 전투 전략을 확장했습니다.

셀 기반 필드는 지형과 속성의 반응 규칙을 전투에 연결한다



ElementCellGrid 설계 의도

- 맵을 셀 단위 데이터로 관리해 지형/속성 상태를 기록
- 투사체가 지나간 위치를 셀 좌표로 변환해 필드 적용
- 풀, 물, 모래 등 지형과 불, 얼음, 바람 속성의 조합 규칙 분리
- 장애물, 벽, 환경 반응을 같은 좌표계로 다룰 수 있게 구성

풀 / 물

기본 필드

불 / 얼음 / 바람

확장 반응

규칙 기반

핵심 가치

속성 레벨과 지형 조합에 따라 서로 다른 반응이 발생한다



풀 + 화염

화염 레벨에 따라 확산 범위와 화염량이 달라짐

모래 + 바람

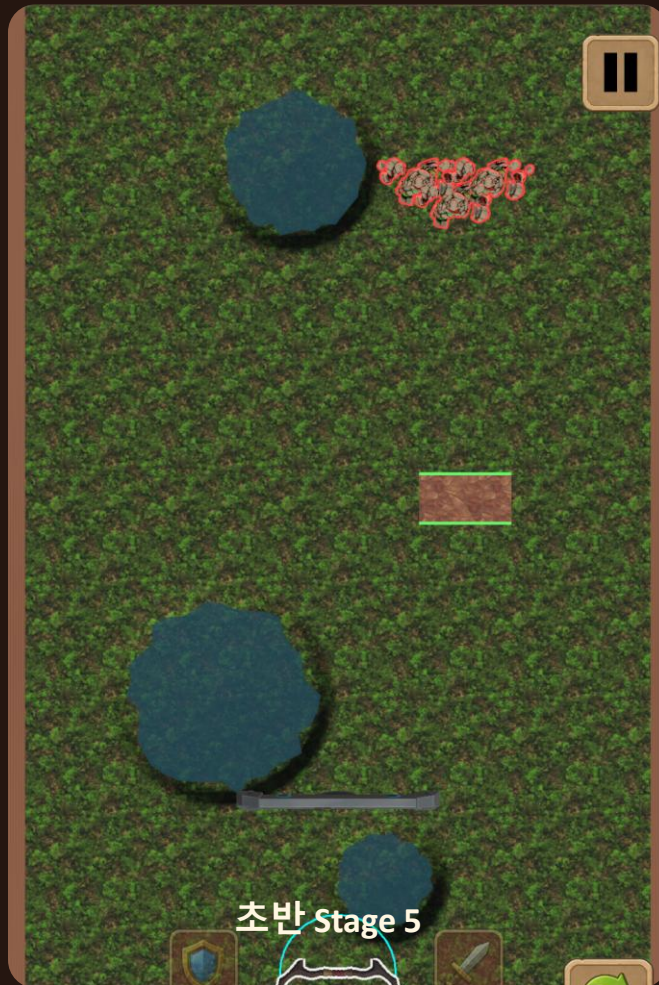
중앙 바람길과 주변 열풍이 퍼지는 특수 반응

물 + 얼음

물 타일 위에서 얼음 속성 판정과 시각 효과 확인

반응이 없는 조합도 규칙으로 관리: 화염+모래, 바람+기타 필드는 의도적으로 추가 반응 없지만, 추후 확장 가능하다.

난이도 값 하나로 몬스터 수, 패턴, 특성을 동적으로 구성했다



보스 / 일반 몬스터

StageManager와 Wave 개념으로 스폰 흐름 구성

Photon 기반 PvP 환경에서 전투 상태를 동기화했다



동기화 범위

- 매칭 후 PvP 씬 전환 및 플레이어 스폰
- 무기 조준 방향과 입력 상태 동기화
- 투사체 생성, 이동, 충돌, 분열 처리
- VFX/SFX, HP, 라운드 상태, 승패 흐름 반영
- 로컬 기준 HUD 표시와 매치 종료 처리

미구현: MMR/랭킹

구현: PvP 전투 환경

뒤끝 서버와 UI를 연결해 계정, 뽑기, 장착 데이터를 관리했다



연동 결과

로그인
↓
뽑기
↓
결과 반영
↓
인벤토리
↓
장착 데이터

일정 관리와 출시.포트폴리오 관점의 우선순위를 점검했다

받은 피드백

- PM 관점에서 일정과 우선순위 정리
- 출시 시 필요한 로그인/계정 흐름 검토
- 포트폴리오에 드러나야 하는 핵심 시스템 방향 제시

반영한 내용

- WBS 기준으로 파트별 산출물과 담당자 정리
- OAuth/간편로그인은 향후 확장으로 분리
- 시연 가능한 전투/PvP/Backend 기능 중심으로 범위 조정

향후 보완

- 간편로그인, 랭킹, MMR 경쟁전
- 출시용 과금/광고 플로우 완성
- QA 케이스와 회귀 테스트 문서화 강화

핵심 플레이 흐름을 기준으로 테스트 관점을 정리했다

입력 영역 검증

공격/방어 영역이 겹치지 않고, 드래그 해제 시 조준·발사·방패 조작이 정상 동작하는지 확인

모바일 양손 조작 GIF로 정상 흐름 확인

투사체/필드 반응

불·얼음·바람 속성과 풀·물·모래 타일 조합에서 의도한 반응만 발생하는지 확인

반응 있음/없음 케이스를 분리해 GIF로 검증

PvP 동기화

두 클라이언트에서 스폰, 조준, 투사체, HP, 라운드 상태가 동일하게 반영되는지 확인

Photon 대전 화면으로 동기화 결과 확인

Backend 데이터

로그인, 뽑기 결과, 인벤토리, 장착 데이터가 UI와 서버 데이터 흐름에 맞게 반영되는지 확인

뽑기-인벤토리 화면으로 데이터 반영 확인

시연한다

1

양손 조작

방패와 무기 영역 분리, 조
준선, 발사

2

필드 반응

속성 레벨과 지형 조합별
반응 차이

3

PvP 대전

Photon 동기화와 라운드
전투

4

Backend

로그인, 뽑기, 인벤토리, 장
착 데이터

기획 대비 핵심 플레이와 시스템 연동은 높은 수준으로 완성했다

완성도 평가

9 / 10

핵심 전투, PvP, Backend, UI 연동
까지 구현했으며 일부 출시 기능
은 향후 확장 범위로 남겼습니다.

잘한 점

- 조작, 전투, 필드, 증강의 시스템 구조화
- Photon PvP 환경 구축
- 뒤끝 기반 데이터 흐름 연동
- 팀원별 파트 분담과 통합 경험

아쉬운 점 / 개선

- 랭킹, MMR, 간편로그인 미구현
- 출시용 과금/광고 플로우 보완 필요
- QA 문서화와 자동 회귀 테스트 강화 필요
- 모바일 빌드 최적화 추가 검증 필요

프로젝트를 통해 기획, 구조 설계, 협업, 네트워크/Backend 연동, 발표용 결과물 정리까지 게임 개발 전 과정을 경험했습니다.